# NURa Hand-in 3

Silvan Toet (s1827332)

April 12, 2026

## 1 Satellite galaxies around a massive central

We once again have the number density profile for satellite galaxies around a massive central (dark matter) halo. The spherical volume element $dV$ can be defined as follows: We know that the volume can be computed in spherical coordinates using the triple integral

$$\iiint_V n(x)dV = \int_{\phi=0}^{2\pi}\int_{\theta=0}^{\pi}\int_{x=0}^{x_{max}} n(x)x^2 \sin\theta\, dx\, d\theta\, d\phi \tag{1}$$

$$= \int_{x=0}^{x_{max}} 4\pi n(x)x^2 dx = \int_{x=0}^{x_{max}} N(x)dx \tag{2}$$

$$= 4\pi A\langle N_{sat}\rangle \int_{x=0}^{x_{max}=5} x^2 \left(\frac{x}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c)dx = \langle N_{sat}\rangle \tag{3}$$

$$= 4\pi A\langle N_{sat}\rangle \int_{x=0}^{x_{max}=5} x^{a-1} \left(\frac{1}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c)dx = \langle N_{sat}\rangle \tag{4}$$

$$-> A = \frac{1}{4\pi \int_{x=0}^{x_{max}=5} x^{a-1} \left(\frac{1}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c)dx} \tag{5}$$

Where we have defined the number of satellites N(x)dx in the infinitesimal range [x, x+dx). Since n(x) where x is the radius relative to virial radius $x = r/r_{vir}$ is only dependent on x, we can easily solve for $\theta$ and $\phi$. Dividing by $\langle N_{sat}\rangle$ on both sides and rearranging gives us an expression for $A$.

The general calling code for all subquestions is included below. Specific functions used for one or several subquestions are shown at that subquestion.

```python
# imports
from collections.abc import Callable
from functools import partial
import matplotlib.pyplot as plt
import numpy as np

from Q1_golden_section_minimizer import bracket_minimum, golden_section_search
from Q1_poisson import negative_poisson_ln_likelihood


def readfile(filename):
    """
    Helper function to read in the satellite galaxy data from the provided text
    files.

```

```python
    Parameters
    ----------
    filename : str
        The name of the file to read in.

    Returns
    -------
    radius : ndarray
        The virial radius for all the satellites in the file.
    nhalo : int
        The number of halos in the file.
    """
    f = open(filename, "r")
    data = f.readlines()[3:]  # Skip first 3 lines
    nhalo = int(data[0])  # number of halos
    radius = []

    for line in data[1:]:
        if line[:-1] != "#":
            radius.append(float(line.split()[0]))

    radius = np.array(radius, dtype=float)
    f.close()
    return (
        radius,
        nhalo,
    )  # Return the virial radius for all the satellites in the file, and the number of halos


def minimize_chi2(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the chi-squared value for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the chi-squared value.
    min_chi2 : float
        The minimum chi-squared value achieved.
    """

    # TODO: implement the minimization of chi2 using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_chi2 = chi2(model, data, initial_params)  # replace by the correct calculation of chi2 for the given parameters

    return best_params, min_chi2


```

```python
def minimize_poisson_ln_likelihood(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the Poisson negative log-likelihood for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the Poisson negative log-likelihood value.
    min_ln_likelihood : float
        The minimum Poisson negative log-likelihood value achieved.
    """

    # TODO: implement the minimization of the Poisson negative log-likelihood using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_ln_likelihood = negative_poisson_ln_likelihood(model, data, initial_params)  # replace by the correct calculation of the Poisson negative log-likelihood for the given parameters

    return best_params, min_ln_likelihood


# ======================================================
# ======== Main functions for each subquestion ========
# ======================================================


def do_question_1a():
    # ======== Question 1a: Maximization of N(x) ========
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    A_1a = 256 / (5 * np.pi ** (3 / 2))
    bounds_for_maximization_and_integration = (0, 5)
    x_lower, x_upper = 10**-4, 5
    x_range = np.linspace(x_lower, x_upper, num=1000)  # TODO: maybe replace by log axis

    fig1a, axs = plt.subplots(1, 2, figsize=(16, 10))
    plt.suptitle("n(x) dx, the number density profile, compared to N(x) dx, \nthe number of satellites in the infinitesimal range [x, x+dx)")
    axs[0].plot(x_range, n(x_range, A_1a, Nsat, a, b, c))
    axs[0].set_title("n(x) dx")
    axs[1].plot(x_range, N(x_range, A_1a, Nsat, a, b, c))
    axs[1].set_title("N(x) dx")
    plt.savefig("Plots/nx_vs_Nx.png", dpi=600)

    # First we want to create a 3 point bracket which brackets our maximum
    # Since our algorithms are designed to find the minimum, we have to input the negative of the function in question
```

```python

    N_1a = partial(N, A=A_1a, Nsat=Nsat, a=a, b=b, c=c)  # create a partial function with all variables that are given for Q1a fixed
    N_1a_negative = lambda x: -N_1a(x)  # Create the negative of the function
    three_point_bracket = bracket_minimum(func=N_1a_negative, a=1, b=1.1)
    print(three_point_bracket)
    x_max, number_of_iterations = golden_section_search(N_1a_negative, *three_point_bracket)
    Nx_max = N_1a(x_max)
    print(f"{x_max=}, {Nx_max=}, {number_of_iterations=}")
    # replace with calculation of the maximum of N(x) based on n(x, A, Nsat, a, b, c) and your minimizer
    # We will use our Golden Section Search minimizer to find the maximum of N(x).

    # Note that I am a bit confused that the template used 1e-4 to 5 as the x range to find the maximum (and did not say to replace the bounds), whereas the question asks to find the maximum between 0 and 5.
    # When I replaced the bounds 0 to 5 in the previous handin for the integration constant calculation to 1e-20 to 5 to solve a 0^x problem in Python, I lost points
    # I have now incorporated the hint that this 0^x problem can be alleviated by combining the x^2 with the x^(a-3) term so I will use the true 0 to 5 x range in my calculation for A this time,
    # Since I don't expect a to go below 1 while fitting the data and give the same problem.

    # Write the results to text files for later use in the PDF
    with open("Calculations/satellite_max_x.txt", "w") as f:
        f.write(f"{x_max:.6f}")
    with open("Calculations/satellite_max_Nx.txt", "w") as f:
        f.write(f"{Nx_max:.6f}")


def do_question_1b():
    # ======== Question 1b: Fitting N(x) with chi-squared ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    N_sat = []
    min_chi2_values = []
    best_params_chi2 = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")
        print(f"{datafile=}, {np.min(radius)=}, {np.max(radius)=} {nhalo=}")

        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data
        bins = 30  # choose appropriate bins

        # TODO: implement the fitting of N(x) to the data using chi-squared minimization.

        # Store N_sat, chi2 values and best-fit parameters in their arrays
        N_sat.append(0.0)
        min_chi2_values.append(0.0)
        best_params_chi2.append((0.0, 0.0, 0.0))  # replace by the correct best-
```

```python
    fit parameters (a,b,c) found from chi-squared minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=bins)  # plot the histogram of the data

        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from chi-squared minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_chi2.png")

    # Save N_sat, chi2 values and best-fit parameters for each data file to tex files for later use in the PDF
    with open("Calculations/table_fitparams_chi2.tex", "w") as f:
        rows = list(zip(N_sat, min_chi2_values, best_params_chi2))
        for idx, (N, chi2_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {N:.5f} & {chi2_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1c():
    # ======== Question 1c: Fitting N(x) with Poisson ln-likelihood ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    min_poisson_llh_values = []
    best_params_poisson = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")
        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data

        # TODO: implement fit using Poisson negative log-likelihood minimization.

        # Store poisson llh values and best-fit parameters in their arrays
        min_poisson_llh_values.append(0.0)
        best_params_poisson.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization

```

```python
        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=10)  # plot the histogram of the data
        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_poisson.png")

    # Save poisson llh values and best-fit parameters for each data file to text files for later use in the PDF
    with open("Calculations/table_fitparams_poisson.tex", "w") as f:
        rows = list(zip(min_poisson_llh_values, best_params_poisson))
        for idx, (llh_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {llh_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1d():
    # ======== Question 1d: Statistical tests ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    G_scores_chi2 = []
    Q_scores_chi2 = []

    G_scores_poisson = []
    Q_scores_poisson = []

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        # Use best-fit parameters from previous steps
        best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
        best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

        # TODO: implement the statistical tests to calculate G and Q scores for both chi2 and poisson fits, and store the results in their respective arrays

        # Append the G and Q scores for chi2 and poisson fits to their respective arrays
        G_scores_chi2.append(0.0)
        Q_scores_chi2.append(0.0)
        G_scores_poisson.append(0.0)
        Q_scores_poisson.append(0.0)

    # Save G and Q scores for chi2 and poisson fits to tex files for later use in the PDF
```

```python
    with open("Calculations/statistical_test_table_rows.tex", "w") as f:
        rows = []
        for i, (G, Q) in enumerate(zip(G_scores_chi2, Q_scores_chi2), start=11):
            rows.append(f"$\\chi^2$ & m{i} & {G:.5f} & {Q:.5f}")

        for i, (G, Q) in enumerate(zip(G_scores_poisson, Q_scores_poisson), start=11):
            rows.append(f"Poisson & m{i} & {G:.5f} & {Q:.5f}")

        for idx, row in enumerate(rows):
            if idx < len(rows) - 1:
                f.write(row + " \\\\\n")
            else:
                f.write(row)


def do_question_1e():
    # ======== Question 1e: Monte Carlo simulations ========
    # pick one of the data files to perform the Monte Carlo simulations on, e.g. m12
    datafiles = ["m11", "m12", "m13", "m14", "m15"]
    index = 1  # index of the data file to use for Monte Carlo simulations, e.g. 1 for m12

    radius, nhalo = readfile(f"Data/satgals_{datafiles[index]}.txt")

    # Use best-fit parameters from previous steps for the original data file
    best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
    best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

    pseudo_chi2_params = []
    pseudo_poisson_params = []

    num_pseudo_experiments = 10  # replace by number with reasonable runtime
    for i in range(num_pseudo_experiments):
        # TODO: generate pseudo-data by sampling from original best-fit chi2 and poisson models
        # Then, for each pseudo-dataset, perform the chi2 and poisson fits to find the best-fit parameters.

        # Append the best-fit parameters for each pseudo-dataset to their respective arrays.
        pseudo_chi2_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization for the pseudo-dataset
        pseudo_poisson_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization for the pseudo-dataset

    # plot the pseudo best-fit profiles, plot the original best-fit profile in another color and plot the mean in one more color

    # chi2 plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_chi2_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from chi-squared minimization

    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from chi-squared minimization on the real
```

```python
    data

    mean_params_chi2 = np.mean(pseudo_chi2_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - chi2 fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_chi2.png")

    # poisson plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_poisson_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from Poisson negative log-likelihood minimization
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization on the real data

    mean_params_poisson = np.mean(pseudo_poisson_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - Poisson fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_poisson.png")


if __name__ == "__main__":
    do_question_1a()
    do_question_1b()
    do_question_1c()
    do_question_1d()
    do_question_1e()
```

```python
# I have moved these functions to a separate file to prevent circular import errors
from collections.abc import Callable

import numpy as np


def n(x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
```

```python
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return A * Nsat * (x / b) ** (a - 3) * np.exp(-((x / b) ** c))


def f(x: np.ndarray, a: float, b: float, c: float) -> np.ndarray:
    return (x / b) ** (a - 3) * np.exp(-((x / b) ** c))


def N(x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> Callable:
    """N(x) dx is the number of satellites in infinitesimal range [x, x+dx).
    It is related to n(x) dx, the number density profile according to N(x) dx = n(x) 4pi x**2 dx."""
    return 4 * np.pi * A * Nsat * x ** (a - 1) * (1 / b) ** (a - 3) * np.exp(-((x / b) ** c))


#### Fitting ####
def get_normalization_constant(a: float, b: float, c: float, Nsat: float) -> float:
    """
    Calculate the normalization constant A (which is a function of a,b,c) for the satellite number density profile.

    Parameters
    ----------
    a : float
        Small-scale slope.
    b : float
        Transition scale.
    c : float
        Steepness of exponential drop-off.
    Nsat : float
        Average number of satellites.

    Returns
    -------
    float
        Normalization constant A.
    """
    # TODO: implement the calculation of the normalization constant
    return 0.0  # replace by the correct value
```

## 1.1 (1a) Maximum of N(x)

For this question we are asked to find the maximum of N(x), the number of satellites in the infinitesimal range [x, x+dx), for given values of a, b, c, $N_{sat}$ and $A$. Let us first examine both functions.

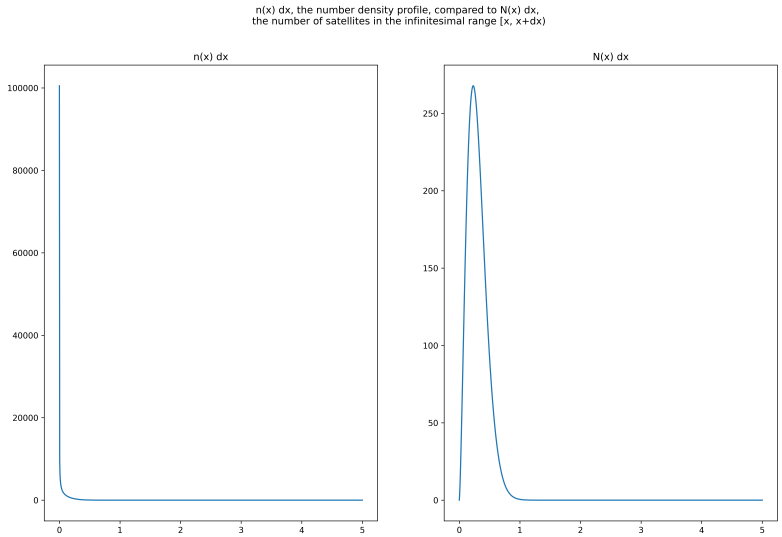


Figure 1: n(x) dx, the number density profile, compared to N(x) dx, the number of satellites in the infinitesimal range [x, x+dx)

Although n(x), the number density profile, is a monotonically decreasing function, because the number density will decrease with radius, N(x) has a maximum because it can first increase at small x while the $x^2$ term gets larger, before the exponential decay dominates at larger radii.

Since this is a one-dimensional minimization problem, we employ a Golden Section Search minimizer together with the unnamed algorithm for bracketing a minimum to find the maximum of N(x). We first define a partial function $N_{1a}$(x) with a, b, c, $N_{sat}$ and $A$ filled in. We then define a function $N_{1a,neg}$ = -$N_{1a}$(x), the negative of the above function. The reason for this is that a minimization algorithm applied to the negative of a function f(x) can find the x coordinate where f(x) is at a maximum. Filling in that x-value in $N_{1a}$(x) gives us the maximum value of the function.

The unnamed bracketing algorithm starts with two x-value points a and b (not to be confused with the a and b constants of n(x)). It then proposes new points until it is certain that the three point bracket a,b,c contains a minimum. To show this algorithm is successful in bracketing a minimum, we start it with a=1, b=1.1, even though we can see from the plot the minimum is around x=0.2. The three point bracket then becomes (-0.5326237921249277, 0.15278640450004133, 0.5763932022500207). Note that the first point is outside the provided x range (0,5). In this case this is not a problem as there are no additional local minima and the function is defined outside the range (0,5), but it is something to always keep in mind when applying this algorithm.

The Golden Section Search minimizer divides this three point bracket into smaller and smaller intervals, ensuring the interval to be self-similar. This means that if we start with two intervals A (between point a and b) and B (between point b and c), we pick a new point d in such a way that A/B = B/C = $\phi$, the golden ratio. Here C is the interval between a and d. For each step we test in which interval the minimum is now contained and adjust the outer edges of our bracket accordingly. This approach ensures best worst-case behaviour.

The maximum of $N(x)$ is found at:

$$x_{\max} \approx 0.229983$$

$$N(x_{\max}) \approx 267.845533$$

### 1.1.1 Code for N(x) maximization

```python
import math
from collections.abc import Callable

import numpy as np


def bracket_minimum(func: Callable, a: float, b: float) -> tuple[float, float, float]:
    """Use the algorithm on lecture 7 slide to find a a 3 point bracket for a function.
    Note that depending on the function and initial guess for a and b, this function is not guaranteed to find a bracket for only 1 minimum!
    There is no way to know this without evaluating the function at many points. This is why we have to restart often.
    """
    phi = (1 + math.sqrt(5)) / 2  # Golden ratio
    fa, fb = func(a), func(b)
    switched_ab = False
    if fb > fa:  # Step 1: ensure f(b) < f(a)
        fa, fb = fb, fa
        a, b = b, a
        switched_ab = True
    c = b + (b - a) * phi  # Step 2
    while True:
        if (fc := func(c)) > fb:  # Step 3a
            return (c, b, a) if switched_ab else (a, b, c)

        # Step 3b: fit a polynomial. for this we can use LU Decomposition of a Vandermonde matrix
        # OR (since the minimum of the polynomial can be analytically defined using a,b,c and fa,fb,fc), we use the formula for this on slide 11 (Brent's method)
        d = b - 0.5 * (b - a) ** 2 * (fb - fc) - (b - c) ** 2 * (fb - fa) / ((b - a) * (fb - fc) - (b - c) * (fb - fa))
        fd = func(d)
        if b < d < c:
            if fd < fc:  # Step 4
                return (b, d, c)
            if fd > fb:
                return (a, b, d)
            d = c + (c - b) * phi
        else:  # Step 5
            d = c + (c - b) * phi
        a, b, c = b, c, d
        fa, fb = fb, fc
```

```python


def golden_section_search(func: Callable, a: float, b: float, c: float, rel_target_acc: float = (np.finfo(float).eps) ** 0.5, max_num_iterations: int = 50) -> tuple[None | float, int]:
    """Iteratively tighten a 3 point bracket surrounding a minimum. This function assumes there is only 1 minimum inside the bracket.
    Use the algorithm defined in lecture 7 slide 10.
    For a minimization algorithm we can only hope to get as close to the minimum as the square root of the
    RELATIVE machine error epsilon times the x-value of the minimum, hence rel_target_acc: float = (np.finfo(float).eps) ** 0.5

    """
    phi = (1 + math.sqrt(5)) / 2  # Golden ratio
    w = 2 - phi
    c_min_b, b_min_a = abs(c - b), abs(b - a)
    other_edge = c if c_min_b > b_min_a else a  # Step 1
    iteration = 1
    while iteration < max_num_iterations:
        d = b + (other_edge - b) * w
        fb, fd = func(b), func(d)
        if abs(c - a) < rel_target_acc * b:  # Step 2. rel_target_acc * b to make it absolute target accuracy!
            if fd < fb:
                return d, iteration
            return b, iteration
        if fd < fb:  # Step 3
            if b < d < c:
                a, b = b, d
                c_min_b, b_min_a = abs(c - b), abs(b - a)  # I need more lines for the various cases,
                other_edge = c if c_min_b > b_min_a else a  # but it is more efficient because I do not always have to recompute both intervals and check which value should become other_edge
            else:
                c, b = b, d
                c_min_b, b_min_a = abs(c - b), abs(b - a)
                other_edge = c if c_min_b > b_min_a else a
        else:  # Step 4
            if b < d < c:
                c = d
                c_min_b = abs(c - b)  # Step 5 optimization: only recompute tightened interval. Here larger interval is b_min_a
                other_edge = a
            else:
                a = d
                b_min_a = abs(b - a)
                other_edge = c
        iteration += 1
    return None, iteration
```

## 1.2 (1b) $\chi^2$-fit

For each halo-mass bin, we want to fit the same parametric profile N(x;a,b,c) separately. This yields five different best-fit parameter sets (a,b,c), one for each mass bin. The normalization A is not an independent fit parameter, but is recalculated from a,b,c for each model using Romberg integration. Because the radial coordinate is normalized by the virial radius, the model describes the dimensionless shape of the satellite distribution, allowing that shape to vary with halo mass.

We are first asked to choose any number of radial bins, real or log space and x range. Since it is not clear from the question if this should vary per file, we assume our choices should be constant over all files (also because these choices are not variables in the template code for this question).

After reading in all files, we calculate the smallest and largest radius of all the satellite galaxies per halo. It turns out that there are no satellite galaxies with an x coordinate smaller than 1e-4, so we choose this as our $x_{min}$ value. The maximum x coordinate is around 2.5, so we simply keep $x_{max} = 5$ as our $x_{max}$ value. We choose logarithmically spaced bins in x to ensure sufficient resolution in the inner regions, where the satellite density varies rapidly, while still sampling the outer regions efficiently. We use 30 bins to balance resolution against Poisson noise, ensuring each bin contains a sufficient number of satellites.

We choose Levenberg-Marquardt as our minimization routine for the fitting procedure, because we are dealing with a model nonlinear in its parameters, we are proud of our LM class implementation and want to try a derivative based method for bonus points.

Since we are fitting our model to binned data, we are comparing the observed mean number of satellites per halo in radial bin i $N_i$, to the model prediction $\tilde{N}_i$ for the mean number of satellites per halo in radial bin i.

Our model is thus

$$\tilde{N}_i = 4\pi\langle N_{sat}\rangle A(a,b,c) I_i(a,b,c) \tag{6}$$

where we define

$$I_i(a,b,c) = \int_{x=i}^{x_{i+1}} x^2 \left(\frac{x}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c) dx = \int_{x=i}^{x_{i+1}} f(x) x^2 dx \tag{7}$$

and

$$I_{total} = \int_{x=0}^{x_{max}} f(x) x^2 dx \tag{8}$$

Note that A is dependent on a, b and c. The partial derivative of $\tilde{N}_i$ to each parameter is thus obtained using the product rule

$$\frac{\partial \tilde{N}_i}{\partial p} = 4\pi\langle N_{sat}\rangle \left(\frac{\partial A(a,b,c)}{\partial p} I_i(a,b,c) + A(a,b,c)\frac{\partial I_i(a,b,c)}{\partial p}\right) \tag{9}$$

Calculating the partial derivatives of f(x) with respect to a, b and c gives

$$\frac{\partial f(x)}{\partial a} = f(x) ln\left(\frac{x}{b}\right) \tag{10}$$

$$\frac{\partial f(x)}{\partial b} = f(x)\left(-\frac{a-3}{b} + \frac{c}{b}\left(\frac{x}{b}\right)^c\right) \tag{11}$$

$$\frac{\partial f(x)}{\partial c} = -f(x)\left(\frac{x}{b}\right)^c ln\left(\frac{x}{b}\right) \tag{12}$$

This gives for all p

$$\frac{\partial I_i}{\partial p} = \int_{x_i}^{x_{i+1}} \frac{\partial f(x)}{\partial p} x^2 dx \tag{13}$$

And using $A = (4\pi I_{total})^{-1}$

$$\frac{\partial A}{\partial p} = -4\pi A^2 \frac{\partial I_{total}}{\partial p} \tag{14}$$

So

$$\frac{\partial \tilde{N}_i}{\partial p} = 4\pi \langle N_{sat} \rangle A \left( \frac{\partial I_i}{\partial p} - \frac{I_i}{I_{total}} \frac{\partial I_{total}}{\partial p} \right) \tag{15}$$

Argue the choices of your binning. Explain the minimization routine used. Bonus points for a correctly implemented derivative-based routine.

Table 1: Best-fit parameters for $\chi^2$ fits.

| Dataset | $N_{\text{sat}}$ | $\chi^2$ | $a$ | $b$ | $c$ |
|---|---|---|---|---|---|
| m11 | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m12 | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m13 | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m14 | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m15 | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |

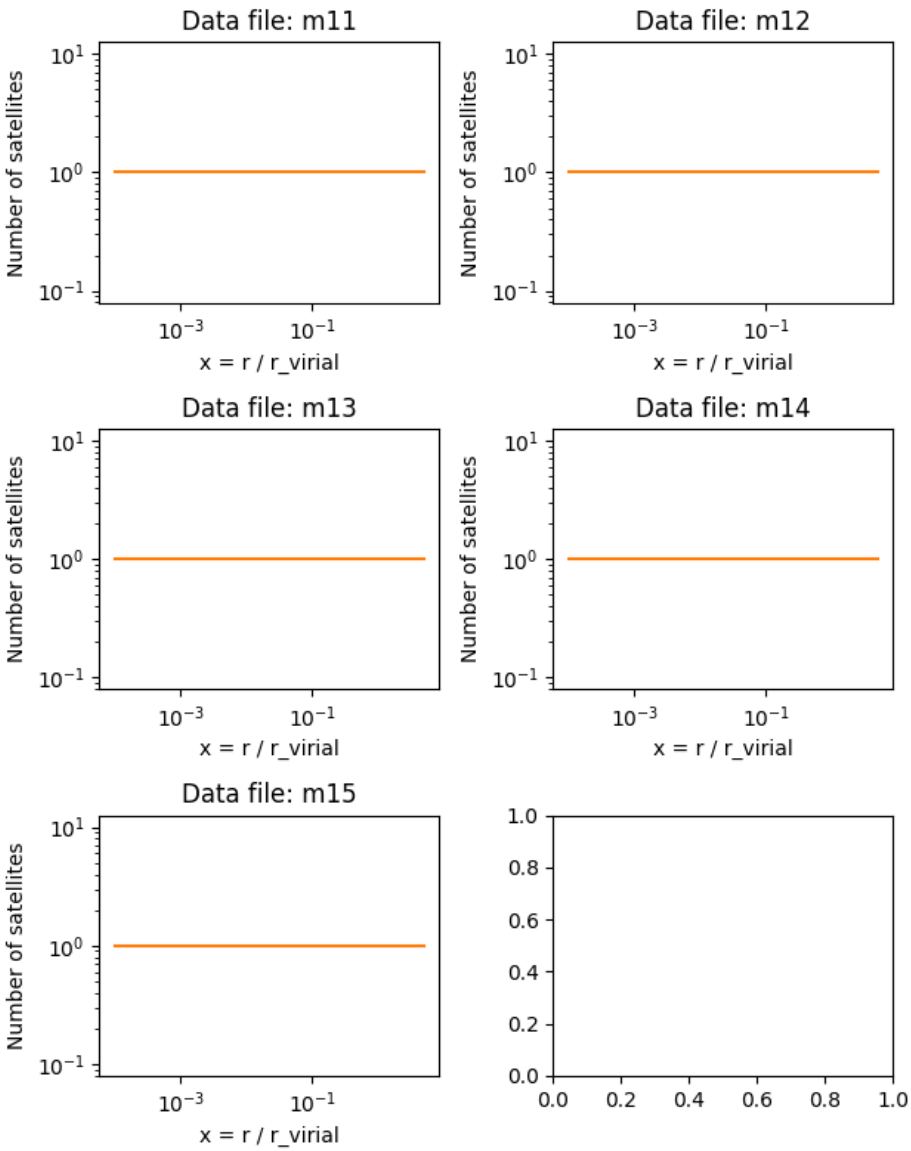


Figure 2: Fit of the satellite number density profile using the chi-squared method.

### 1.2.1 Code for $\chi^2$-fit

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

```
def chi2(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the chi-squared for a given set of parameters and data.

    Parameters
    ----------
```

```
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The chi-squared value for the given parameters and data.
    """
    # TODO: implement calculation of the chi2 value (or equivalent) using the model mean and variance to be minimized.

    return 0.0  # replace by the correct value
```

### 1.3 (1c) Poisson ln-likelihood fit

Explain how you determine the Poisson ln-likelihood. Describe the optimization method used. How do your best fit parameters differ from the ones obtained in (1b)? Why?

Table 2: Best-fit parameters for $\chi^2$ fits.

| Dataset | llh | $a$ | $b$ | $c$ |
|---|---|---|---|---|
| m11 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m12 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m13 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m14 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| m15 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |

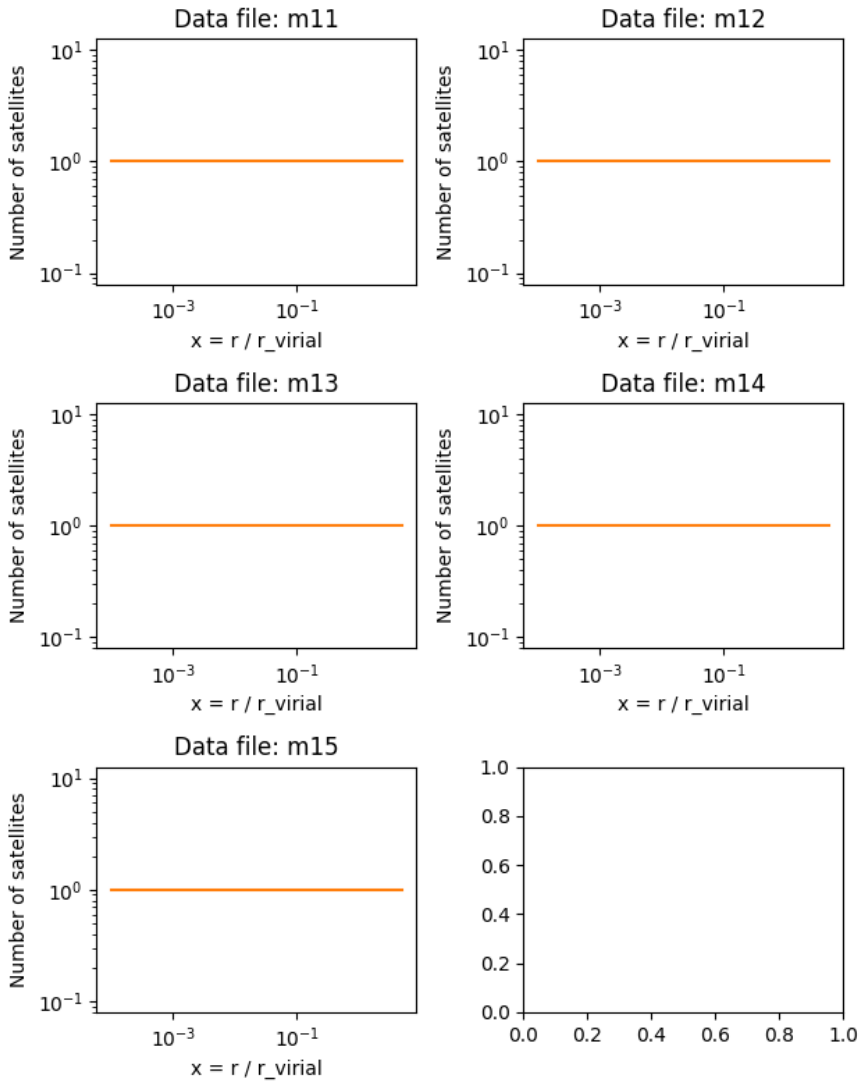


Figure 3: Fit of the satellite number density profile using the Poisson log-likelihood method.

### 1.3.1 Code for Poisson ln-likelihood fit

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

```
import numpy as np


def negative_poisson_ln_likelihood(model: callable, data: np.ndarray, params:
    tuple) -> float:
    """
    Calculate the Poisson negative log-likelihood for a given set of parameters
    and data.

```

```
    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The Poisson negative log-likelihood value for the given parameters and data.
    """
    # TODO: implement calculation of the Poisson negative log-likelihood (or equivalent) to be minimized

    return 0.0  # replace by the correct value
```

## 1.4 (1d) Statistical tests

Table 3: Statistical tests for the $\chi^2$ and Poisson ln-likelihood fits.

| Method | Dataset | G | Q |
|---|---|---|---|
| $\chi^2$ | m11 | 0.00000 | 0.00000 |
| $\chi^2$ | m12 | 0.00000 | 0.00000 |
| $\chi^2$ | m13 | 0.00000 | 0.00000 |
| $\chi^2$ | m14 | 0.00000 | 0.00000 |
| $\chi^2$ | m15 | 0.00000 | 0.00000 |
| Poisson | m11 | 0.00000 | 0.00000 |
| Poisson | m12 | 0.00000 | 0.00000 |
| Poisson | m13 | 0.00000 | 0.00000 |
| Poisson | m14 | 0.00000 | 0.00000 |
| Poisson | m15 | 0.00000 | 0.00000 |

What can you conclude from the results of the statistical tests?

### 1.4.1 Code for statistical tests

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

## 1.5 (1e) Bonus Monte Carlo simulation

Pick a single dataset. Describe the implementation of your Monte Carlo parametric bootstrap.

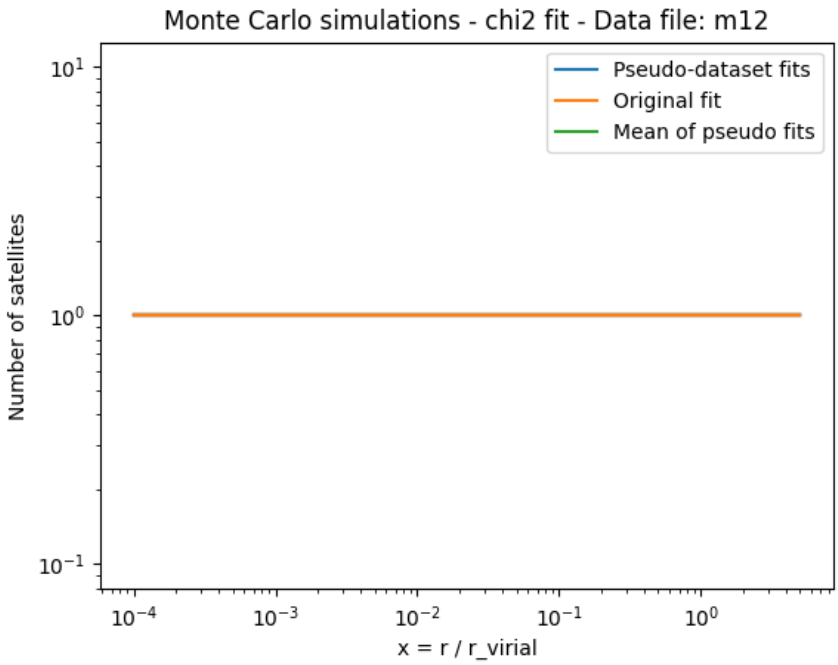


Figure 4: Best $\chi^2$-fit profiles using pseudo-experiments.

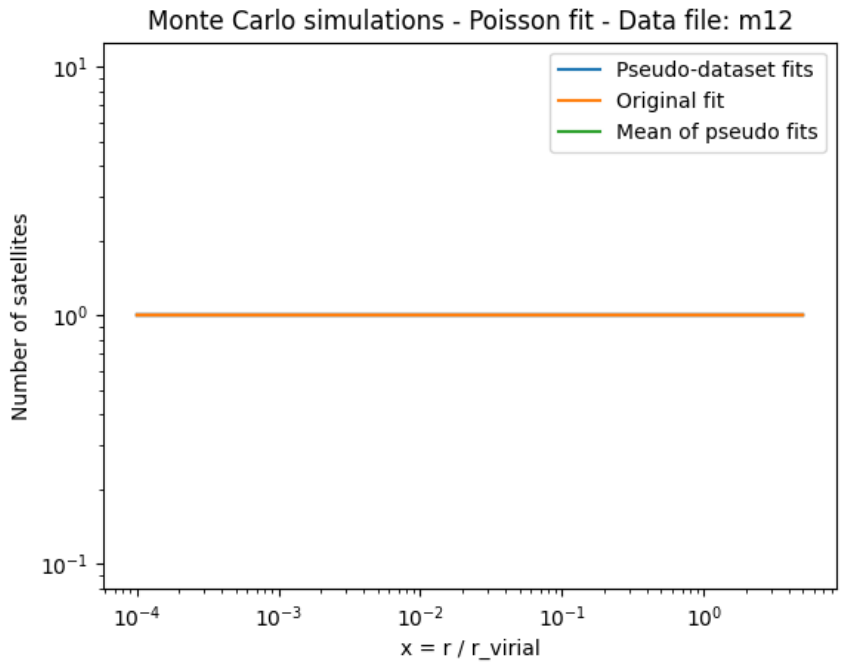


Figure 5: Best Poisson-likelihood fit profiles using pseudo-experiments.

### 1.5.1 Code for Monte Carlo simulation

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.